Reviewer Pack — Minimal Rank of Geometric Invariants Bounds Monotone Circuit…

Entry adc629fc7d9b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 18:09:43 UTC

Minimal Rank of Geometric Invariants Bounds Monotone Circuit Size

Entry ID: `adc629fc7d9b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 18:09:43 UTC

1. Conjecture

Field A (mathematical branch): Geometric Invariant Theory (GIT) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> {'quantitative': 'For every n-vertex graph G, let ν(G) be the minimal rank of a projective variety V with a morphism φ: V → Gr(n, 3) that satisfies |Im(φ)| = |E(G)|. Then the monotone circuit size for computing the Tutte polynomial T_G(x,y) is Θ(2^(ν(G)))', 'falsifier': 'There exists an n-vertex graph G with ν(G) ≤ k such that the minimal monotone circuit size for computing T_G(x,y) is less than 2^k'}

Rationale (proposer's reasoning):

> {'explanation': 'Geometric Invariant Theory (GIT) provides tools to study properties of varieties invariant under algebraic group actions. By associating a projective variety V to a graph G and finding the minimal rank of such a variety, we can capture structural information about G that is relevant to its computational complexity.', 'connection': 'The Tutte polynomial encodes many combinatorial properties of graphs, including circuit complexity. The conjecture bridges the gap by proposing a quantitative relation between GIT invariants and Boolean function complexity.'}

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b2ab35f8dd56e0d3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n-vertex graphs G with ν(G) ≤ k, the computed monotone circuit size for T_G(x,y) is at least 2^k with 95% confidence.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Geometric Invariant Theory' AND 'Boolean Circuit Complexity' AND Tutte polynomial' - 'monotone circuit size' AND 'minimal rank of projective variety' AND morphism to Gr(n,3)' - 'computing Tutte polynomial with minimal monotone circuits' AND 'projective variety with morphism φ: V → Gr(n, 3)'

Top relevant hits considered: - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2004.12063v2] Hardness of Random Optimization Problems for Boolean Circuits, Low-Degree Polynomials, and Langevin Dynamics - [http://arxiv.org/abs/1204.3563v4] On the Tutte-Krushkal-Renardy polynomial for cell complexes - [http://arxiv.org/abs/2411.02936v3] Conditional Complexity Hardness: Monotone Circuit Size, Matrix Rigidity, and Tensor Rank - [http://arxiv.org/abs/0705.3525v1] Comment on "Minimal size of a barchan dune" - [http://arxiv.org/abs/1210.4781v2] Finite Morphisms between Projective Varieties and Skeleta - [http://arxiv.org/abs/1708.08132v1] Topological Tutte Polynomial - [http://arxiv.org/abs/1906.02781v2] Tutte Polynomial Activities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.add((i, j))
        return edges

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for i in range(n):
            max_row = rank
```

```
            for j in range(rank, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            if A[max_row][i] == 0:
                continue
            A[rank], A[max_row] = A[max_row], A[rank]
            for j in range(n):
                if j != i:
                    factor = A[j][i] / A[rank][i]
                    for k in range(n):
                        A[j][k] -= factor * A[rank][k]
            rank += 1
        return rank

    def compute_tutte_polynomial(G, x, y):
        n = len(G)
        M = [[0] * (n + 2) for _ in range(n + 2)]
        M[0][0], M[0][1], M[1][0] = 1, -x, -y
        for u, v in G:
            M[u+1][v+1], M[v+1][u+1] = 1, 1

        return gaussian_elimination(M)

    def min_rank(G):
        n = len(G)
        max_edges = n * (n - 1) // 2
        for k in range(1, max_edges + 1):
            for edges in itertools.combinations(G, k):
                if len(edges) == k:
                    M = [[0] * (k + 2) for _ in range(k + 2)]
                    M[0][0], M[0][1], M[1][0] = 1, -x, -y
                    for u, v in edges:
                        M[u+1][v+1], M[v+1][u+1] = 1, 1
                    if gaussian_elimination(M) == k + 2:
                        return k + 2
        return max_edges

    n = random.randint(5, 40)
    G = generate_random_graph(n)
    x, y = random.uniform(-10, 10), random.uniform(-10, 10)

    ν_G = min_rank(G)
    circuit_size = compute_tutte_polynomial(G, x, y)

    return {
        "metric_name": "circuit_size",
        "metric_value": circuit_size,
        "instances_tested": 1,
        "conjecture_holds": circuit_size >= 2 ** ν_G,
        "counterexample": "" if circuit_size >= 2 ** ν_G else f"Graph with n={n}, ν(G)={ν_G}, circ
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7bee5763.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7bee5763.py", line 75, in run_trial
    ν_G = min_rank(G)
          ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7bee5763.py", line 66, in min_rank
    M[u+1][v+1], M[v+1][u+1] = 1, 1
    ~^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means we cannot verify the conjecture's support or falsification under the given conditions. | next: Re-run the test ensuring it completes successfully and produces the required data to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11948
2	preregistration	ollama_remote	glm4:latest	0	0	5919
3	novelty	ollama_remote	glm4:latest	0	0	5258
4	novelty	ollama_remote	glm4:latest	0	0	10435
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21481
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13256
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10208
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11537
9	judge	ollama_remote	glm4:latest	0	0	9162

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99205 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/adc629fc7d9b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/adc629fc7d9b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/adc629fc7d9b.tar.gz` (if generated)